LeakLens: Detecting Group and Temporal Data Leakage in Machine Learning Code by Linking Static Analysis to an Instrumented Run

Srijeet Prasad Banerjee, Adrish Sarkar, Ankur Roy,
Sayandeep Dey, Soumyarik Mukherjee and Sayan Hazra
Department of Information Technology, *Guru Nanak Institute of Technology*, Kolkata, India

***Abstract*—When evaluation data reaches model training, a reported score stops predicting how the model will do on new data. Code analysers for machine learning catch preprocessing, overlap and multi-test leakage, but the leaks that distort results most, across patients or across time, are either missed or found only when an identifier column is named in the code. We present LeakLens, a prototype for Python scripts and notebooks. Its static pass reports six leakage types and takes evidence for an entity from column names, file paths, attributes of objects and hand-written splits; an optional instrumented run checks entity and time overlap at the split, fold by fold, with nothing declared by the user; and a located leak can be fixed in a copy of the pipeline and re-run. On the held-out half of an injected benchmark (79 files, 73 leaks), scored once, with a group finding counted only when it names the true entity, the static pass finds 59 leaks with no false positive and the combined mode 61 with four. Per file, the analyser of Yang et al. is right for 20 of 37 files and LeakLens for 33 ($p < 0.001$). On real code it flags the random-window and within-patient regimes of an EEG study and leaves the leave-one-patient-out regime alone, and on public notebooks it names the patient column of a patient-level leak and confirms a shuffled forecasting split whose fix lowers the reported R^2 from 0.96 to 0.89. Impact estimates are preliminary: fixes that change the split also change the rows being scored.**

***Index Terms*—data leakage, static analysis, dynamic analysis, Jupyter notebooks, machine learning engineering, reproducibility**

I. Introduction

A held-out score means something only while the held-out rows stay out of model building. *Data leakage* is any route by which evaluation rows, or information that will not exist at prediction time, reach training [1]. Leaky code runs without errors and passes its tests; the only sign is a score that is better than it should be.

The problem is widespread. A review of earlier surveys counted 294 affected papers in 17 scientific fields, and in one re-analysis the advantage of complex models over logistic regression vanished after the errors were corrected [2]. Applied to 107,603 public notebooks on GitHub, a static analyser reported a leak in 29.6% of them [3].

The kinds of leak differ a great deal in cost. Across 2,047 tabular datasets, scaling all rows before the split, the example every course uses, moved AUC by at most 0.005, while leakage across group and time boundaries did not show up under random cross-validation at all [4]. Of 63 translational EEG papers, only 17 deep learning studies in the survey clearly kept each subject on one side of the split, and holding subjects out in one experiment took accuracy from 99.8% down to 53.0% [5].

Code analysers concentrate on the cheap kind. The analyser of Yang et al. [3] and the tools built on it [6]–[8] report preprocessing, overlap and multi-test leakage. Two recent, unreviewed linters flag a random split when an identifier column such as `patient_id` is named in the code [9], [10], and one LLM-based linter covers part of temporal leakage [11]. None of them sees an entity the code leaves unnamed: a subject code inside a file name, an attribute of a recording object, or a column that is only in the loaded table. Data-level checkers find group and date overlap [12]–[14], but only after the user exports the tables and names the column, and their answer does not point to a line of code. None of these tools measures how much a leak changes the user's own result.

LeakLens[1] is our attempt to close these gaps. It contributes:

1) **Group leakage found from code without relying on a named column**: entity evidence from column names, file-path identifiers and attributes of items, and recognition of hand-written random and per-entity splits. Temporal leakage is tracked through data flow, an improvement on existing code checks rather than a first.
2) **A link from static findings to an instrumented run**: the static pass nominates the split call; the run checks entity and time overlap there, per fold, and finds identifier columns the code never mentions.
3) **A per-finding impact estimate**, measured by re-running the user's own pipeline with a canonical fix, including notebooks that only display their score. We present it as preliminary.
4) **An evaluation in three parts**: an injected benchmark of 175 files (162 leaks) split before any rule was written; five case studies on real code; and a protocol for 79 real notebooks labelled independently by two people.

II. Background and Related Work

Taxonomies. Leakage was formalised by Kaufman et al. [1]; Kapoor and Narayanan [2] distinguish eight types, and Roth [4] measured severity by type.

[1]An unrelated package with the same name exists on PyPI. The name of this project may change before any release.

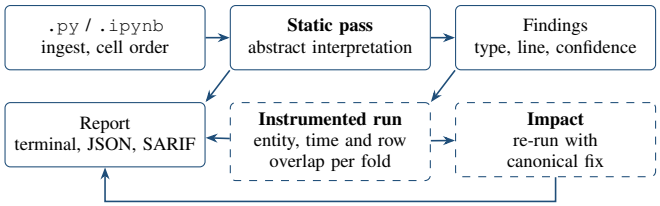


Fig. 1. Stages of LeakLens. Dashed stages execute the code and are optional.

Static analysis. Yang et al. [3] model data flow in Datalog and report three types (precision 97.6%, recall 67.8% on hand-labelled notebooks). NBLyzer and later abstract-interpretation work verify that training and test data stay apart [15]–[17]. LeakageDetector puts the analyser of Yang et al. into an IDE with quick fixes [6], [7], and Draoui et al. [8] add LLM prompts to static slicing; their abstract, which is all we could read, reports F1 gains of 22% for preprocessing and 15% for overlap. scicode-lint [11] asks a local LLM pattern questions, two of them about temporal leakage. leakray and unleak [9], [10], released in 2026 without a paper, look for identifier names such as `patient_id` near a random split and never run the code.

Checks on data and at run time. Deepchecks [12] checks duplicates and date overlap between supplied tables; newer packages add group overlap [13], [14], [18]. splitguard [19] watches splits at run time for group identifiers the user registers, and ArgusEyes [20] tracks row provenance.

Measuring the effect. LeakHound [14] estimates inflation with a proxy model trained on the given split; mlwhatif [21] re-runs pipelines for data-cleaning questions; Leakly [22] re-runs a pipeline on permuted labels, which signals leakage somewhere without locating it. In LeakLens an estimate belongs to one located finding and uses the user's own pipeline and metric. Table I sets LeakLens next to the analyser of Yang et al., the closest prior work.

III. Leakage Taxonomy

LeakLens reports six types (Table II): the three of Yang et al. [3], group and temporal leakage (non-independence and temporal leakage for Kapoor and Narayanan [2]) and target-derived features. Whether a group finding is a mistake depends on the claim: a model meant for the patients it was trained on may share patients across a split. The tool cannot know the claim, so group findings carry a confidence and can be suppressed inline with a reason.

IV. Design of LeakLens

Fig. 1 shows the three stages; only the static stage leaves the code unexecuted.

A. Static pass

Notebooks are read in execution order when every code cell has a unique execution count, otherwise in cell order. One pass of abstract interpretation gives each variable an abstract value: where a data frame came from, its known columns, which statistics were computed over which rows, whether rows were copied, and whether it is the training or evaluation side of a split. Rules fire at sinks: at a split (library splitters, cross-validation helpers, search objects, manual samples) the analyser checks for group and temporal leakage and reports statistics or copied rows that reached the data being split; at a fit, statistics that include evaluation rows; at a score, a model fitted on the scored rows; and a selection over scores computed in a loop gives a multi-test finding.

```
ri, wi, y = training_windows(recs, ...)
u = rng.random(len(ri))
tr, va, te = u < f_tr, (u >= f_tr) & (u < f_tr + f_va), u
    >= f_tr + f_va
info = learner.fit_windows(recs, (ri[tr], wi[tr], y[tr]),
    ...)
zt = learner.logits_windows(recs, ri[te], wi[te]) - corr
subj = np.array([recs[k].subject for k in ri[te]])

line 99 [medium] group (LL-GRP-001): rows split without
    regard to
  'subject' (attribute of the items of 'recs').
  fix: GroupShuffleSplit with groups='subject'
```

Listing 1. Random split of 1-second EEG windows pooled over patients (shortened from the case study code) and the static finding.

Evidence for an entity is a column or variable name, an identifier cut from a file path, or an attribute with an entity name read on the items of a collection (`recs[k].subject`). Strong names such as `patient` or `subject` can give medium confidence, weak ones such as `customer` or `visit` only low. Research code often splits by hand, so the analyser also treats a uniform random draw compared with a threshold (`u = rng.random(n); tr = u < 0.8`) as a row-level split once both sides index the same data, and a loop that sends the first rows of every entity to one list and the rest to another as a within-entity split. Functions that a file defines but never calls are analysed once with unknown arguments. Listing 1 shows a case from the EEG study of Section VII.

The analyser would rather miss a leak than invent one: an unmodelled call depends on all its inputs and anything unresolved produces no finding. Loops are analysed once, functions are inlined to a fixed depth, and classes are opaque. pandas, NumPy, scikit-learn [23] and imbalanced-learn are modelled by name; catalogues and thresholds live in configuration files.

B. Instrumented run

Some entities never appear in the code, for example a `patient_id` column in a table that is split at random. The dynamic pass runs the script in a child process with scikit-learn's splitters, cross-validation helpers, fits, predictions, scores and metric functions wrapped. At each split and fold it records rows hashed by value on both sides; candidate entity columns in the script's row-aligned data frames, reported when their values repeat and enough evaluation rows share an entity with training rows; and, for a time column, whether training rows are dated after the earliest evaluation row. These facts are joined to the static findings by call site, so each finding ends up *confirmed*, *refuted* or *not checked*; leaks only the run can see, such as duplicated raw records, are added.

TABLE I

DESIGN COMPARISON BETWEEN THE STATIC ANALYSER OF YANG ET AL. AND LEAKLENS. ENTRIES FOR YANG ET AL. COME FROM THEIR PAPER AND OUR RUNS OF THEIR RELEASED TOOL. THE TWO ACCURACY ROWS WERE MEASURED ON DIFFERENT DATA WITH DIFFERENT LABELLING AND ARE NOT COMPARABLE; TABLE IV IS THE LIKE-FOR-LIKE MEASUREMENT.

Aspect	Yang et al. [3]	LeakLens (this work)
Analysis technique	Static data-flow analysis written in Datalog (Soufflé), with type inference	One-pass abstract interpretation over the Python syntax tree, linked to an optional instrumented run
Runs the analysed code	No	Optional (dynamic and impact modes only)
Leakage types	Preprocessing, overlap, multi-test	The same three, plus group, temporal and target-derived features
Group and temporal leakage	Not types it claims	Group: entity from column names, file paths, attributes of items, or a column found at run time; confirmed at the split. Temporal: non-chronological splits, forward-looking features
Location reported	The affected evaluation line (as observed in our runs)	The line that causes the leak, with a suggested fix
Impact on the score	Not estimated	Re-runs the pipeline with a canonical fix (six kinds); preliminary
Evaluation data	100 hand-labelled real notebooks (94 analysed) and 107,603 public GitHub notebooks	175 synthetic files with injected leaks; five real-code case studies; 79 real notebooks being labelled
Accuracy reported by the authors	Precision 97.6% (40/41), recall 67.8% (40/59), real notebooks	Recall 0.81 (static, no false positive) and 0.84 (with the run, 4 false positives), own injected TEST split, correct entity required
Time per file	Mean 3.23 s (their paper); median 6.2 s on our files (Kaggle CPU)	Static: median 2.6 ms. Instrumented run: median 1.3 s (laptop)

TABLE II

THE SIX LEAKAGE TYPES AND THE SIGNAL USED BY EACH PASS.

Type	Static signal	Run-time check
Preprocessing	A transformer, selector or encoder is fitted on rows that later reach the evaluation side	The fit included evaluation rows
Overlap	Resampling or concatenation before the split; evaluation rows flow into a fit	Identical rows on both sides of the split
Multi-test	Several candidates scored on the same evaluation rows feed a selection	Same rows scored repeatedly
Group	A row-level or per-entity split on data with a candidate entity	The same entity value on both sides
Temporal	A shuffled split on time-ordered data; forward-looking feature	Training rows dated after evaluation rows
Target-derived	A feature computed from the label	Very high single-feature score

C. Impact estimation

Given a located finding, LeakLens runs the script again with a single call site altered. Six fixes exist: drop duplicated records before a split; make a row-level split group-aware on the detected column; make a split chronological; remove evaluation rows from a fit; refit a transformer on training rows only; drop a feature. The estimate is the original metric minus the fixed one, negated for metrics where lower is better, so a positive value always means the leak flattered the result. The metric is the last `name: value` line the script prints or, when there is none, the last score the run itself produced at a call site both runs reach (a metric call, `.score`, a cross-validation helper, or a number a notebook cell displays). Findings without a fix, or whose fixed run fails, are reported as "impact unknown"; if the fixed run takes a different path, the estimate is marked unreliable.

D. Untrusted code, output and tests

Notebooks from the internet run only in an offline cloud session (Kaggle), in a child process with an allow-listed environment, a throwaway directory, a timeout and an unprivileged user, so that it cannot read the session's tokens. Findings (location, type, rule, confidence, evidence, proposed fix, and the before/after metric where measured) are written as text, JSON or SARIF, with a pre-commit hook and a GitHub Action. The tool is deterministic and uses no language model. Of its 383 tests, 381 pass on a laptop; two need root and run only in the cloud.

V. BENCHMARK AND EVALUATION PROTOCOL

A. Injected benchmark

No public labelled dataset includes group and temporal leakage, so we built one. Thirteen clean pipelines in three families (tabular, time series, grouped data such as multi-visit patients and EEG-style windows) are mutated by 18 operators. Seventeen inject a leak, for example a scaler fitted before the split or a shuffled split on time-ordered data; one produces a *trap*, code that looks leaky but is correct. The result is 175 files (94 scripts, 81 notebooks) with 162 leaks, 13 clean files and 6 traps. Leaky lines are known by construction and every leaky file has a corrected twin, so true inflation can be measured. Temporal leaks fall into *common*, *differentiating* (outside existing pattern checks, such as the default shuffle of `train_test_split`) and *hard* cases. The benchmark was split by base pipeline into DEV (7 pipelines, 96 files, 89 leaks) and TEST (6 pipelines, 79 files, 73 leaks) before any rule existed; rules were tuned on DEV.

B. Protocol

The protocol was frozen before any TEST result existed. A finding matches a label when it is in the same file and cell, its lines overlap the labelled lines and its type is the labelled one or a listed alternative; matching is one-to-one, and the operating point is medium confidence or above. Recall is reported per type with 95% bootstrap intervals over files. After an internal review we added one condition, logged with its reason and applied to both looks: a group finding counts only if it names the true entity, or a variable assigned directly from it; otherwise it is a false positive and the label a miss. TEST is scored once per release; later runs are labelled second looks. True inflation is the leaky file's metric minus its twin's, averaged over five data seeds; the generators were built so that each leak type can move the metric, so these values score estimates only and say nothing about severity in practice.

C. Baselines

Yang et al. We built the released analyser [3] from source and ran its default command, with its Datalog engine at version 2.4.1 because the packaged one no longer installs. Because it points at the evaluation call rather than the line that leaks, we compare the two tools file by file and type by type, a rule fixed before any baseline output on TEST existed.

Data-level and run-time tools. leakcheck-ml [13], LeakHound [14], dataleaks [18] and Deepchecks [12] read tables, not code. Our instrumented run exported, at every split and for up to five folds, the tables the pipeline used, and each tool saw every pair with its defaults and again with the true entity or time column named; a detection is placed at the split line. splitguard [19] was run as documented and with the true group declared. All ran at the cited versions after the single TEST run of LeakLens.

Not run. scicode-lint [11] needs a GPU with native FP8 support; on two NVIDIA T4s it gave no result for any of the 29 files it attempted. LeakCheck [8] needs a paid LLM service and an access-restricted package. leakray and unleak [9], [10] appeared after our TEST run and have not yet been run.

VI. Results on the Injected Benchmark

A. Detection on the held-out split

Table III gives the single TEST run under the strict entity rule. The static pass finds 59 of 73 leaks with no false positive; the instrumented run adds both duplicated-record leaks and reaches 61, but it also turned four group misses into four findings that name the wrong column, so it has four false positives. Under the frozen rule alone, which checked only type and line, the combined mode had scored 7 of 7 group leaks and 65 in total; we consider that figure overstated. The run confirmed 43 static findings, refuted none and could not check 25. By temporal category the tool found 5 of 5 common, 7 of 7 differentiating and 8 of 13 hard leaks. On DEV, where rules were tuned, it found 80 of 89 statically; the drop to TEST is real. Per file, the static pass took a median of 2.6 ms and the run 1.3 s (at most 12.7 s) on a laptop, against 6.2 s for the Yang analyser in a cloud job.

B. What went wrong, and a second look

Three faults explain most misses. In a patient-visits pipeline the static pass guessed a weak column, `visit`; the run checked only that column, found visit numbers on both sides as always, and raised its confidence without looking at `patient_id`. Trimming the last row with `data.iloc[:-1]` looked like a manual split and hid five temporal leaks and one multi-test leak, and a `startswith` column filter that the analyser could not resolve hid two more. We fixed the three faults, logged the change and ran TEST again: 67 of 73 statically and 73 of 73 with the run, all group findings naming the right entity, no false positive. We wrote these fixes with the failing TEST files in front of us, so the second look is diagnostic only and Table III stays the result of record.

TABLE III
Detection on the injected TEST split (79 files, 73 leaks), scored once; a group finding counts only if it names the true entity. Medium confidence or above; recall with 95% bootstrap interval over files.

		Static		Static + run	
Type	**Leaks**	**Found**	**Recall**	**Found**	**Recall**
Preprocessing	22	21	0.95 (0.84–1.00)	21	0.95 (0.84–1.00)
Overlap	9	7	0.78 (0.44–1.00)	9	1.00 (1.00–1.00)
Multi-test	6	5	0.83 (0.43–1.00)	5	0.83 (0.43–1.00)
Group	7	3	0.43 (0.00–0.83)	3*	0.43 (0.00–0.83)
Temporal	25	20	0.80 (0.62–0.95)	20	0.80 (0.62–0.95)
Target-derived	4	3	0.75 (0.00–1.00)	3	0.75 (0.00–1.00)
All types	73	59	0.81	61	0.84
False positives		0		4*	

*Four further group findings are at the right line but name `visit` instead of `patient_id`; the frozen line-and-type rule had counted them, giving 7 and 65.

TABLE IV
Yang et al. and LeakLens (static) on the 79 injected TEST files, scored per file and type. Found: files with the leak in which the tool reports the type. False: files where it reports the type wrongly.

		Yang et al. [3]		LeakLens (static)	
Leakage type	**Files**	**Found**	**False**	**Found**	**False**
Preprocessing	22	10	1	21	0
Overlap / duplicates	9	7	0	7	0
Multi-test	6	3	0	5	0
Group†	7	0	0	3	0
Temporal†	24	3	0	19	0
Target-derived†	4	0	0	3	0

†Not a type the baseline claims. It is credited under temporal in 3 files where it reports whole-series scaling as preprocessing. Temporal has 24 files for 25 leaks.

C. Comparison with Yang et al. on the same files

On the three types the baseline supports (Table IV) it was right for 20 of 37 files with one false alarm, and the static pass of LeakLens for 33 with none; both analysed all 79 files and stayed silent on the leak-free ones. Over the 37 file-and-type pairs, only LeakLens was right in 14, only the baseline in 1, and 19 were right for both and 3 for neither (exact McNemar test, $p < 0.001$). Preprocessing makes the difference (12 against 1); for overlap the two tools give identical answers. The setting favours LeakLens: neither tool was tuned on these files, but they come from our operators, which target the leaks LeakLens was built for.

D. Data-level and run-time tools at a split

Table V covers the TEST leaks a data-level tool can see: 8 non-chronological splits of time-ordered data, 7 group splits and 2 sets of duplicated records. With no column named, leakcheck-ml finds 4 of 7 group splits, all with the right column, and 3 of 8 time splits; LeakHound, Deepchecks and splitguard find neither. Told the column, leakcheck-ml and LeakHound reach 7 of 8 and 7 of 7. dataleaks finds 7 of 7 group splits unaided but names an unrelated column in 3 and flags 2 of 9 leak-free files. LeakLens finds 8 of 8 time

TABLE V
LEAKS AT A SPLIT CALL, INJECTED TEST. COL.: TRUE COLUMN NAMED TO THE TOOL. T1: NON-CHRONOLOGICAL SPLITS; GRP: GROUP SPLITS (LINE RULE), COL.: NAMING THE TRUE ENTITY; DUP.: DUPLICATED RECORDS; CLEAN: LEAK-FREE FILES FLAGGED.

Tool	Col.	T1 /8	Grp /7	Grp, col. /7	Dup. /2	Clean /9
LeakLens (static + run)	no	8	7	3	2	0
leakcheck-ml	no	3	4	4	2	0
	yes	7	7	7	2	0
LeakHound	no	0	0	0	2	0
	yes	7	7	7	2	0
dataleaks	no	0	7	4	2	2
	yes	0	7	7	2	2
Deepchecks	no	0	0	0	2	0
	yes	7	0	0	2	0
splitguard	no	0	0	0	0	0
	yes	0	7	7	0	0

splits with no column named; on group splits it names the true entity in 3 of 7, against 4 for leakcheck-ml without a hint. Every other tool also needs exported tables, which our run supplied here.

E. Impact estimates (preliminary)

For the first look, the result of record, an estimate exists for 34 of 70 located leaks (group 7 of 7, target-derived 3 of 3, overlap 7 of 9, preprocessing 10 of 21, temporal 7 of 25, multi-test 0 of 5). For 26 findings in files with a single leak and a bounded metric, averaged over five seeds, the mean absolute error against true inflation was 0.035 and the rank correlation 0.58. The sign was wrong for all hold-out group splits (-0.084 against a true $+0.212$ AUC on average) because the fix grouped by the wrong column. LeakHound, given the exported splits (Table VI), was more accurate where both tools gave an estimate (11 findings, mean absolute error 0.065 against 0.086).

Two checks bear on how to read these numbers. The direction is consistent: every estimate is oriented so that a positive value means the leak flattered the score, and across 395 runs instrumentation never changed the metric a script printed. The scored rows are another matter. We recorded the evaluation rows of every original and fixed run: the 18 fixes that leave the split alone (refit on training rows, drop a feature, remove evaluation rows from a fit) score identical rows, while all 16 fixes that change the split score different rows, with a mean overlap of 0.14 to 0.28 by fix type. For those, the estimate is the change under the canonical fix, not a like-for-like difference. In the second look, which is diagnostic only, the error fell to 0.0006 and the group estimates had the right sign.

VII. REAL CODE

A. Case studies

For each case we report the mechanism, what LeakLens says, whether it names the right column, the corrected pipeline and the change in the metric (Table VII).

TABLE VI
IMPACT ESTIMATES AGAINST TRUE INFLATION (TEST, SINGLE-LEAK FILES, SEED 0). COMP.: PIPELINES REPORTING AUC OR R^2. CELLS: ESTIMATES (MEAN ABSOLUTE ERROR). LH: LEAKHOUND; COL.: TRUE COLUMN NAMED.

Type	N	Comp.	LeakLens	LH	LH, col.
group	6	6	6 (0.158)	0 (–)	6 (0.075)
overlap	6	6	6 (0.000)	2 (0.043)	2 (0.043)
preproc.	8	8	8 (0.000)	0 (–)	0 (–)
target-der.	3	3	3 (0.000)	0 (–)	0 (–)
temporal	6	3	3 (0.000)	0 (–)	3 (0.059)

EEG. CalibraEEG, a seizure-detection project by the first author, scores the same models under three regimes on 23 patients: random 1-second windows pooled over patients (A), a chronological split inside each patient (B) and leave-one-patient-out (C). Its splits are hand-written NumPy masks and per-subject loops over recording objects, and our first version of LeakLens found nothing in its 78 files. The rules for hand-written splits and item attributes (Section IV-A) were added after that miss, while looking at this code, so this case is a development case, not a blind one. LeakLens now flags A (Listing 1) and B on `subject` and reports nothing in C. The corrected pipeline is regime C, written by the CalibraEEG authors; with LightGBM the paired AUPRC falls from 0.779 (A) and 0.595 (B) to 0.364 (C).

Public notebooks. To avoid choosing cases LeakLens happens to handle, we fixed a selection rule first: walk the sampling pool of our real-notebook study in its seeded order, skip the study notebooks, and take the first two notebooks per theme whose code shows patient-level or chronological leakage, uses scikit-learn and can run on a CPU. All 253 notebooks inspected are logged with the reason they were taken or skipped (in most medical datasets every patient has a single row), and the labels were written before LeakLens was run on the four chosen. They ran only in the offline sandbox, and only data paths were adapted. Statically, LeakLens names `patient_id` at the random split of the Parkinson's progression notebook (C2b), finds the shuffled splits of both forecasting notebooks only at low confidence, below the reporting threshold, and misses the gait notebook (C2a), whose patients exist only as file names inside a loop over `os.listdir`. In the sales notebook (C3b) the run confirmed that 99% of training rows were dated after the first test row, and the chronological fix lowered the random forest's test R^2 from 0.959 to 0.895 and raised its cross-validated RMSE from 108.6k to 258.0k. It also confirmed the multi-test selection of the best of three models on the test rows, but placed it at the scoring call rather than the selection line. C3b's first run exposed a bug (a printed descriptive statistic was taken as the metric); the fix is logged and leaves the injected results unchanged. The runs of C2a, C2b and C3a need competition data that our Kaggle account could not yet attach.

B. Real notebooks

From 150 public Kaggle notebooks sampled in three strata, the 79 by TEST-side authors form the test set (71 are DEV).

TABLE VII
REAL-CODE CASE STUDIES. ENTITY: TRUE ENTITY OR TIME COLUMN NAMED. Δ: METRIC WITH THE LEAK MINUS METRIC WITH THE FIX.

Case	Leak	Static	Run	Entity	Δ
EEG A	pooled windows, random mask	group	–	yes	+0.415[a]
EEG B	split inside each patient	group	–	yes	+0.231[a]
EEG C	leave-one-patient-out	none	–	n/a	–
C2a	gait samples, random	miss	·	no	·
C2b	patient rows, random	group	·	yes	·
C3a	daily energy, random	low[b]	·	–	·
C3b	weekly sales, random	low[b]	conf.	yes	+0.065[c]

[a]AUPRC against regime C, measured by CalibraEEG itself (LightGBM, 23 patients); the run was not used. [b]Found at low confidence, below the reporting threshold. [c]Test R^2 of the notebook's random forest, chronological fix. ·: run pending (competition data).

The 79 TEST notebooks are being labelled by two authors working independently, with a written guide, a practice round and no access to tool output or AI assistants; agreement will be reported as Cohen's kappa and disagreements settled before LeakLens and the baselines are scored. At the time of writing the labelling is in progress; until then we make no claim about precision or recall on real notebooks.

VIII. Limitations and Threats to Validity

Real code. The benchmark and the tool share authors, and TEST uses new pipelines but the same mutation operators as DEV, so it measures generalisation across pipelines, not across the ways people write leaks. The case studies are few and one of them shaped the rules. **Small samples.** TEST has 4 target-derived, 6 multi-test and 7 group leaks. **Blindness.** TEST column names were seen by mistake while rules were written; no TEST code, label or operator was read. **Entity inference** is heuristic: names, repetition, file paths and attributes can mislead, as the first TEST run shows. **Coverage.** Custom classes and deep learning frameworks remain largely outside the analysis. **Impact estimates** exist for about half of the located leaks and, for split-changing fixes, compare different evaluation rows. **Baselines.** The Yang analyser was scored under a looser rule; the data-level tools used tables exported by our run; scicode-lint, LeakCheck, leakray and unleak were not run.

IX. Conclusion and Future Work

LeakLens reads machine learning code for six leakage types, takes entity evidence from more than column names, checks its findings at the split while the code runs, and can re-run a pipeline with a fix. On held-out injected leaks, counting a group finding only when it names the right entity, it found 81% statically and 84% with the run, more than the reference static analyser on the types they share ($p < 0.001$), and it was the only tool to find the time-order leaks without being told the time column. The review made two weaknesses plain: the run once attached group findings to the wrong column, and impact estimates for split-changing fixes compare different rows. Next we will finish the real-notebook evaluation, run the new linters and scicode-lint, and measure how often each leakage type occurs in public notebooks and how much it costs.

Data and code availability. The code, benchmark generators, frozen protocol, change log and the output of every cited run are released under the MIT licence at https://github.com/bytes06runner/LeakLens.

Acknowledgment

The implementation was written with the help of an AI coding assistant; the authors made the design and protocol decisions and reviewed all reported results.

References

[1] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, 2012.

[2] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, 2023.

[3] C. Yang, R. A. Brower-Sinning, G. A. Lewis, and C. Kästner, "Data leakage in notebooks: Static detection and better processes," in *Proc. 37th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2022.

[4] S. Roth, "Which leakage types matter? A quantitative landscape across 2,047 benchmark datasets," arXiv:2604.04199, 2026.

[5] G. Brookshire *et al.*, "Data leakage in deep learning studies of translational EEG," *Front. Neurosci.*, vol. 18, p. 1373515, 2024.

[6] E. A. AlOmar, C. DeMario, R. Shagawat, and B. Kreiser, "LeakageDetector: An open source data leakage analysis tool in machine learning pipelines," arXiv:2503.14723, 2025.

[7] Truong *et al.*, "LeakageDetector 2.0: Analyzing data leakage in Jupyter-driven machine learning pipelines," arXiv:2509.15971, 2025.

[8] T. Draoui, M. W. Mkaouer, and C. Newman, "Improving data leakage detection in machine learning notebooks through static slicing and structured LLM prompts," *Proc. ACM Softw. Eng.*, vol. 3, no. FSE, 2026.

[9] tpong-dev, "leakray," https://github.com/tpong-dev/leakray, 2026, software, version 0.2.0.

[10] johnnyjosta-go, "unleak," https://github.com/johnnyjosta-go/unleak, 2026, software, version 0.1.0.

[11] Samsonau, "scicode-lint: Detecting methodology bugs in scientific Python code with LLM-generated patterns," arXiv:2603.17893, 2026.

[12] S. Chorev *et al.*, "Deepchecks: A library for testing and validating machine learning models and data," *J. Mach. Learn. Res.*, vol. 23, no. 285, 2022.

[13] "leakcheck-ml," https://github.com/Sofienguy1/leakcheck, 2026, software, version 0.3.0.

[14] J. Gomes, "LeakHound," https://github.com/happyhellpt/leakhound, 2026, software, version 0.5.0.

[15] P. Subotić, L. Milikić, and M. Stojić, "A static analysis framework for data science notebooks," in *Proc. 44th Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2022.

[16] P. Subotić, U. Bojanić, and M. Stojić, "Statically detecting data leakages in data science code," in *Proc. 11th ACM SIGPLAN Int. Workshop on the State of the Art in Program Analysis (SOAP)*, 2022.

[17] F. Drobnjaković, P. Subotić, and C. Urban, "An abstract interpretation-based data leakage static analysis," in *Theoretical Aspects of Software Engineering (TASE)*, ser. LNCS. Springer, 2024.

[18] K. Rajput, "DataLeaks," https://github.com/KAVYA-29-ai/Dataleaks, 2026, software, version 0.1.2.

[19] T. Aiello, "splitguard," https://github.com/Tommasoaiello13/splitguard, 2026, software, version 0.1.0.

[20] S. Schelter, S. Grafberger, S. Guha, B. Karlaš, and C. Zhang, "Proactively screening machine learning pipelines with ArgusEyes," in *Companion of the 2023 Int. Conf. Management of Data (SIGMOD)*, 2023.

[21] S. Grafberger, P. Groth, and S. Schelter, "Automating and optimizing data-centric what-if analyses on native machine learning pipelines," *Proc. ACM Manag. Data (SIGMOD)*, 2023.

[22] DeMONLab-BioFINDER, "Leakly: Leakage checks for machine-learning pipelines using permutation tests," https://github.com/DeMONLab-BioFINDER/Leakly, 2026, software, version 0.1.2.

[23] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.